Aaron Jones

CS 499: Computer Science Capstone — Milestone Four: Enhancement Three, Databases

Artifact Description

The artifact enhanced in this milestone is the Grazioso Salvare Rescue Animal Dashboard, a Python application built on PyMongo and Dash. It was originally created in 2025 for CS 340: Client/Server Development (term C-6). The artifact consists of two files: CRUD_Python_Module.py, which defines an AnimalShelter class that handles the application's interaction with a MongoDB database, and ProjectTwoDashboard.ipynb, a Jupyter notebook that builds an interactive web dashboard on top of it. The dashboard loads real rescue-animal intake and outcome data from the Austin Animal Center into MongoDB, then lets a user filter animals by rescue-type profile (water rescue, mountain or wilderness rescue, or disaster and individual tracking), view the filtered results in a sortable, filterable data table, see a pie chart of breed distribution for the current filter, and see the location of a selected animal on an interactive map.

Justification for Inclusion and Summary of Improvements

I selected this artifact for the Databases category because it is the piece of coursework in my program that most directly demonstrates a full client-to-database pipeline: querying a NoSQL database through a driver, shaping that data for an interactive front end, and working with a real, messy operational dataset rather than a small set of hand-crafted test records. The specific components that showcase my database skills are the AnimalShelter class's query construction, where each rescue-type filter builds a different MongoDB query combining equality and range conditions, and the way the dashboard's callbacks tie that querying directly to what the user sees and interacts with.

The code review completed earlier in this course identified hardcoded database credentials, baked directly into the source code, as the primary issue with this artifact. I used that finding as the starting point but broadened the enhancement to cover the database layer more completely, since a handful of other real problems became apparent once I looked closely at the code. The artifact was improved in four concrete ways. First, credentials are now read from environment variables instead of being hardcoded, and the class fails immediately with a clear error if they are missing rather than silently falling back to a default account. Second, I completed the CRUD module itself: despite its name, the original only implemented Create and Read, with no Update or Delete at all. I added both, and deliberately built in a safeguard so that an empty query is rejected rather than silently applying an update or delete to the entire collection. Third, I added a compound index on the four fields the dashboard's filters actually query together, so a filter click no longer forces MongoDB to scan every document in the collection. Fourth, I fixed three real bugs along the way: the original connection code never actually validated credentials at startup because PyMongo connects lazily, so a bad password would be silently accepted until the first real query failed; the username and password were placed directly into the connection string unescaped, which would break on certain special characters; and the dashboard's logo-loading code had no error handling, so a missing image file crashed the entire application before it could start.

Course Outcomes

In my Module One ePortfolio plan, this artifact was aimed primarily at Outcome 4, using well-founded tools and techniques to deliver a working software or database improvement, and Outcome 5, developing a security mindset that anticipates adversarial exploits and mitigates design flaws, since the code review's central finding was a security issue rather than a performance or design one. Outcome 4 is now fully demonstrated: the completed CRUD operations, the indexing work, and the credential-handling changes are all working, tested code rather than a plan. Outcome 5 is the more significant update to my coverage. Through Milestone Two and Three I had identified a security finding for this category but had not yet implemented a fix for one, so Outcome 5 remained the outcome I had made the least actual progress on. This enhancement is the first time I have a concrete, tested example for it: removing hardcoded credentials, fixing the authentication check that was silently accepting bad credentials, and building in safeguards against an accidental mass update or delete are all direct, defensive responses to real vulnerabilities rather than theoretical ones. I would not say Outcome 5 is fully demonstrated after a single artifact's credential fix, since a complete security mindset would also cover things like encryption in transit and at rest or least-privilege database roles, which this enhancement did not touch, but it is meaningfully further along than it was.

Reflection on the Enhancement Process

The most useful technical lesson from this enhancement was discovering that the original connection code's error handling did not actually do what it looked like it did. PyMongo's MongoClient connects lazily, so wrapping the client constructor in a try/except, which is what the original code did, never actually tests whether the credentials are valid; that only happens on the first real operation. Forcing an explicit ping inside the same try/except was a small code change, but understanding why it was necessary required digging into how the driver actually behaves rather than just how the code reads. Designing the compound index was a similar kind of lesson: getting real benefit out of it meant thinking through which fields the dashboard's queries actually combine and in what order, not just adding an index to every field that gets filtered on.

The biggest challenge was less about the code itself and more about the gap between the managed course environment I originally built this artifact in and a fresh local setup. The original notebook depended on a Jupyter-in-the-browser package that turned out to be effectively abandoned and incompatible with a current Jupyter install, which meant replacing it with the equivalent, actively maintained functionality built into current Dash before I could even get the dashboard running again. Getting my own local MongoDB instance ready for a real end-to-end test surfaced its own chain of gaps: no Jupyter installed, no MongoDB shell or import tools installed, and access control never configured in the first place. None of that reflects a problem with the enhancement itself, which I verified thoroughly through automated testing against the real shelter dataset before ever touching my local machine, but it was a genuine reminder that an artifact that worked perfectly in a preconfigured classroom environment can still need real setup work to run anywhere else, and that is its own kind of lesson about the assumptions baked into a piece of software.

AI Usage Acknowledgment: Generative AI (Claude) was used to help implement the planned database enhancement, identify and fix related bugs, test the implementation against real

data, and draft language for this narrative. The decisions about enhancement scope and all final wording are my own.